Reinforcement Learning in Large Language Models

Lior Shani

Google Research

Roadmap

Google Research

- **Training Language Models**
  - Pre-training (MLE) $\rightarrow$ Post-training (SFT & RL)
- **RL from Human Feedback (RLHF)**
  - KL Regularization, Reward Models & Hacking, DPO, and Preference-Based RL (Nash-MD)
- **RL with Verifiable Rewards (RLVR)**
  - Verifiable Rewards, Critique & Rubrics, Emergent Reasoning (DeepSeek-R1), and KL Revisited
- **Credit Assignment in LLMs**
  - Bandit vs. MDP, Value Baselines, PRMs vs. ORMs, Multiple Rollouts, and GRPO
- **Beyond Vanilla RL**
  - ReST, DAPO, MaxRL ($\mathrm{pass@}k$), Test-Time Compute Scaling, and On-Policy Distillation
- **Multi-turn & Agentic RL**
  - Conversation Preferences (MTPO), Multi-turn Credit Assignment, ReAct/Reflexion, and Open Questions

Training Language Models

Pre-training

Google Research

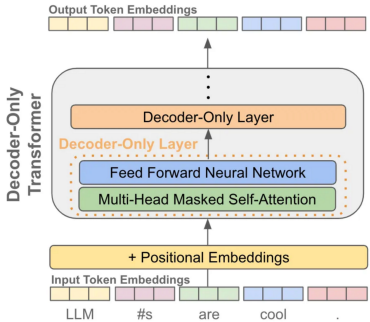
A language model is a machine that predicts probabilities over a vocabulary

- **Goal:**
  - Model the human language/knowledge as a whole
  - Learn a distribution of the textual world
- **How? Pretraining:**
  - Crawl the whole open-source human corpus ($\sim$ 10T Tokens)
  - Train a model using supervised learning loss (next slides)
- **Comments:**
  - Mathematically this is supervised learning, but it is referred to as *self-supervised*
  - In practice, things are a bit more involved, but this is the main picture

Architecture

Google Research

- **Auto-regressive LLMs predict next-token probabilities:**
  - x_t: the vocabulary token at position t
  - p_θ: the language model
  - $x_{<t} = (x_1, \cdots, x_{t-1})$: the sequence up to t (excluding)
  - Predicts $p_\theta(x_t \mid x_{<t})$ for all tokens x_t in the vocabulary
- **Architecture:**
  - A transformer-based autoregressive decoder-only model



original post

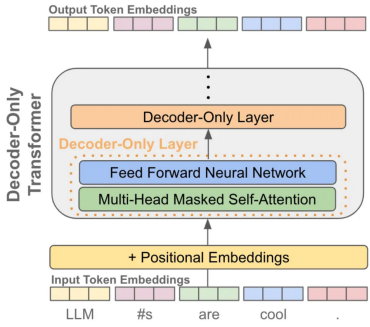
Learning the Language Model

Google Research

- **Objective:**
  - Supervised learning loss (MLE)
  - Minimize cross-entropy for all tokens x_t:

$$-\log p_\theta(x_t \mid x_{<t})$$

- **Teacher-forcing:**
  - The whole sequence is used both for input and target



original post

Post-training: From a Language Model to an Agent

Google Research

The pretrained model is fine-tuned for the required behavior

- **Conceptually:**
  - **Pre-training:** distribution of the textual world
  - **Post-training:** policy / agent
- **Phases:**
  - **SFT (Supervised Fine-Tuning)**
    - Behavior Cloning
    - Examples of how the agent should act/answer for a given context/query
  - **RL (Reinforcement Learning)**
  - *Teacher Model distillation is sometimes used*

RL from Human Feedback (RLHF)

Regularization

Google Research

- **Vanilla RL starts from scratch**
  - Random LLM initialization will lead us nowhere due to high dimensionality
- **How can we continue learning on top of a pretrained model?**
- **Alternative A: Just do RL on top of the model**
  - *Issue:* post-training tasks are not as wide as pre-training $\implies$ we might lose model capability
- **Alternative B: Regularization**
  - KL-loss with respect to the model at step 0 (pretrained/std model)
  - Similar to what's done in offline RL / imitation learning instantiation: *Improve but stay close*
    - **TD3+BC**
    - **BRAC:** $\max_{\pi_\theta} \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}} \left[\mathbb{E}_{a''\sim\pi_\theta(\cdot|s)}[Q_\psi(s,a'')] - \alpha \hat{D}\Big(\pi_\theta(\cdot \mid s), \pi_b(\cdot \mid s)\Big) \right]$
    - **BC+SAC:** $\mathbb{E}_{s,a\sim\pi}[Q(s,a) + \mathcal{H}(\pi(\cdot \mid s))] + \lambda \mathbb{E}_{s,a\sim\mathcal{D}}[\log \pi(a \mid s)]$

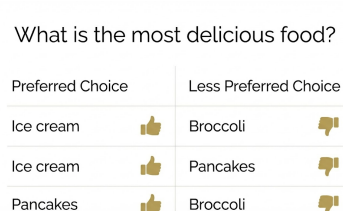
Preferences Feedback

Google Research

RL relies on optimizing a reward

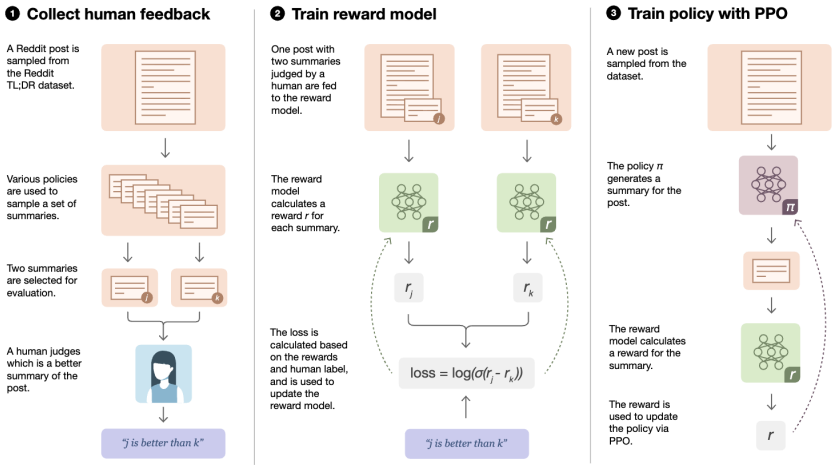
- **Problem:** Reward specification is hard
- **Solution:** Collect preferences data





Reinforcement Learning from Human Feedback

Google Research



RLHF: Preference Data

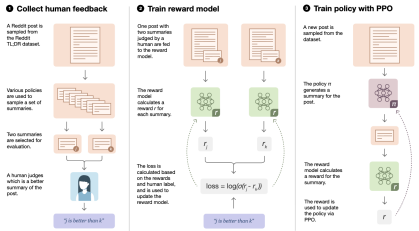
Google Research

Data structure:

- A context (task) x
- Two alternative generations (episodes) y_1, y_2
- **Label:** Which is preferred of the two?

Important:

- Generations should be **on-policy**



Stiennon et al. (2020)

Reinforcement Learning from Human Feedback (RLHF)

Google Research

1. **Collect Preferences Data**
   - Gather human preference comparisons over pair-wise model outputs.
2. **Learn Reward Model** $r(x, y)$
   - **Bradley-Terry Model Assumption** (preferences fit underlying rewards):

$$\mathcal{P}(y \succ y' \mid x) = \frac{1}{1 + \exp\big(r(x, y') - r(x, y)\big)} = \sigma\big(r(x, y) - r(x, y')\big)$$

   - **Cross-Entropy Loss** (optimizes reward gradient):

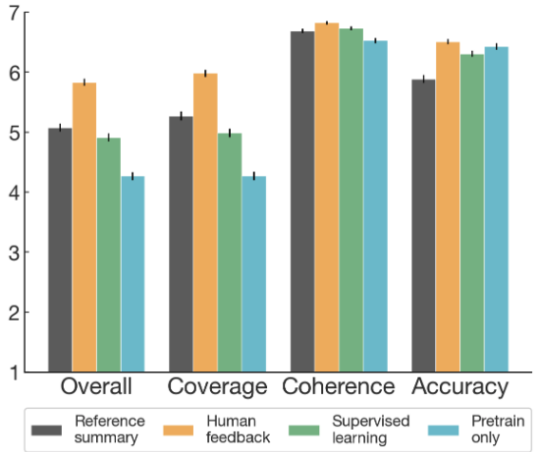
$$\text{loss}(r_\theta) = -\mathbb{E}_{(x, y_0, y_1, i) \sim \mathcal{D}} \Big[\log \sigma\big(r_\theta(x, y_i) - r_\theta(x, y_{1-i})\big) \Big]$$

3. **Optimize Policy Using Vanilla RL**

$$R(x, y) = r_\theta(x, y) - \beta \log \Big[\pi_\phi^{\text{RL}}(y \mid x) \,/\, \pi^{\text{SFT}}(y \mid x) \Big]$$

RLHF Empirical Evidence

Google Research



RLHF Empirical Evidence: Scaling & Overoptimization

Google Research

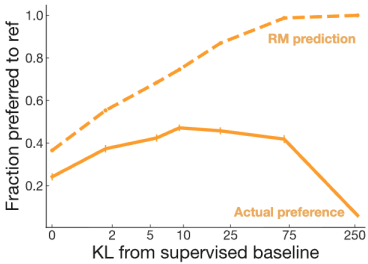


Figure 5: Preference scores versus degree of reward model optimization. Optimizing against the reward model initially improves summaries, but eventually overfits, giving worse summaries. This figure uses an earlier version of our reward model (see rm3 in Appendix C.6). See Appendix H.2 for samples from the KL 250 model.

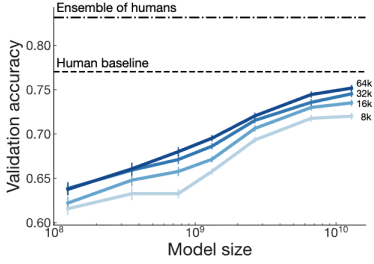


Figure 6: Reward model performance versus data size and model size. Doubling amount of training data leads to a ~1.1% increase in reward model validation accuracy, whereas doubling the model size leads to a ~1.8% increase. The 6.7B model trained on all data begins approaching the accuracy of a single human.

Classifier-Based Reward Model

Google Research

- Same methodology for any classification-based reward model

Size	Method	COMPREHENSION	ATTRIBUTION	SALIENCE	CONCISENESS
XXL	SL	**99.0** ± 1.1	27.3 ± 5.0	61.6 ± 5.5	35.0 ± 5.4
	Filtered	96.3 ± 2.1	31.3 ± 5.2	61.3 ± 5.5	34.3 ± 5.3
	RLEF_{L}	98.7 ± 1.3	**56.6** ± 5.6	**78.0** ± 4.7	**61.0** ± 5.5
	RLEF_{H}	98.0 ± 1.5	39.0 ± 5.5	70.6 ± 5.1	45.3 ± 5.6
Base	RLEF_{H}	96.0 ± 2.2	**38.3** ± 5.5	**64.3** ± 5.4	**44.3** ± 5.6
	CLIFF	**99.3** ± 0.9	28.3 ± 5.1	58.3 ± 5.6	33.3 ± 5.3
XSum	reference	99.3 ± 0.9	23.6 ± 4.8	62.6 ± 5.4	30.3 ± 5.2

Table 3: Human evaluation results over 100 test set samples, each summary rated by 3 workers, results are micro-averaged. Each value corresponds to a percentage of positive answers per category with 95% confidence intervals around the sample proportion. Highest values for each model size are in bold.

	Summary before the NLI flip	Summary after the NLI flip	NLI	Description
1	Two astronauts who spent a year living on the International Space Station have landed in Florida.	Two astronauts who spent a year living on the International Space Station have returned to Earth.	✓	Abstractive Rephrasing
2	Afghan forces have repelled an advance by Taliban fighters on the northern city of Kunduz, officials say.	Afghan forces have been battling Taliban insurgents in the northern city of Kunduz.	✓	Abstractive Rephrasing
3	A senior Nigerian military official has said militant Islamist group Boko Haram is no longer a threat, after a mosque attack that left at least 82 people dead.	A senior Nigerian official has denied that Islamist militant group Boko Haram was behind a mosque attack in the north in which more than 100 people were killed.	✓	Claim Change
4	Two people have been arrested on suspicion of manslaughter after a three-year-old boy died at a water park.	Two people have been arrested after a four-year-old boy died at a water park.	✓	Argument Omission
5	Bolton Wanderers manager Lee Trotter has apologised after he and striker Aaron Lennon swore at fans on live television.	Bolton Wanderers manager Lee Trotter has apologised after he and team-mate Gary Caldwell swore at fans on live television.	✗	Argument Change

Table 4: Examples of summaries for the same document on consecutive checkpoints during RL training, before and after the NLI classification of the summary has flipped. The summaries maintain a stable main structure which enables manual inspection. The main changes are highlighted in gray, the NLI column depicts the entailment decision for the latter summary, and the description specifies the type of the semantic change.

Factually Consistent Summarization via Reinforcement Learning with Textual Entailment Feedback (Roit et al., 2022)

Reward Hacking

Google Research

- **Goodhart's Law:** *"When a measure becomes a target, it ceases to be a good measure"*
  - Policy optimizes the **proxy reward** instead of the **true reward**, hurting generalization
- **Unconstrained extrapolation**
  - The reward model assigns rewards even in low-confidence, out-of-distribution regions
- **Spurious correlations:** Reward models capture superficial patterns in training data
  - E.g., *length bias*: lengthy responses look smarter and are preferred
  - Penalizing length yields overly short responses that omit key information
- **Sim-to-real gap**
  - In simulated environments, models exploit unrealistic simulator quirks that fail in reality

Mitigating Reward Hacking: Ensembles & Pessimism

Google Research

- **Reward Model Ensembles** *(Coste et al., 2023)*
  - Train an ensemble $\{r_1, \ldots, r_k\}$ and optimize a conservative reward objective:
  - **Worst-Case (WCO):** $\min_i r_i(x, y)$ or **Uncertainty-Weighted (UWO):** $\bar{r}(x, y) - \lambda \operatorname{Var}_i\big(r_i(x, y)\big)$
- **Ensembles Mitigate, But Do Not Eliminate Hacking** *(Eisenstein et al., 2023)*
  - Reward models are *underspecified* OOD; pre-training diversity helps more than fine-tuning diversity
  - Ensembles still fail when all members share systematic biases (e.g., length or formatting heuristics)
- **Pessimistic Preference / Reward Optimization (P3O / PRPO)** *(Gupta et al., 2025)*
  - Standard KL regularization penalizes policy drift blindly, independent of data coverage
  - Optimizes against the *worst-case* reward/preference model consistent with offline data $\mathcal{D}$

Offline RLHF: Direct Preference Optimization

Google Research

- **Offline approach for learning from preferences**
  - Does not require sampling with a model
  - Can re-use SFT pipelines
  - Much cheaper to run
- **No need to learn a reward / preference model**

Direct Preference Optimization: Derivation

Google Research

- **The RLHF objective:** $\max_{\pi_\theta} \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi_\theta(y|x)}\left[r_\phi(x,y)\right] - \beta D_{\mathrm{KL}}\left[\pi_\theta(y \mid x) \,\|\, \pi_{\mathrm{ref}}(y \mid x)\right]$
- **Can be solved in closed form:** $\pi_r(y \mid x) = \dfrac{1}{Z(x)}\pi_{\mathrm{ref}}(y \mid x)\exp\left(\dfrac{1}{\beta}r(x,y)\right)$
- **Simple algebra:** $r(x,y) = \beta \log \dfrac{\pi_r(y \mid x)}{\pi_{\mathrm{ref}}(y \mid x)} + \beta \log Z(x)$
- **By Bradley-Terry:** $p^*(y_1 \succ y_2 \mid x) = \sigma\big(r^*(x,y_1) - r^*(x,y_2)\big)$
- **Plugging in the reward:** $p^*(y_1 \succ y_2 \mid x) = \dfrac{1}{1+\exp\left(\beta \log \frac{\pi^*(y_2|x)}{\pi_{\mathrm{ref}}(y_2|x)} - \beta \log \frac{\pi^*(y_1|x)}{\pi_{\mathrm{ref}}(y_1|x)}\right)}$
- **MLE gives the following loss:**

$$\mathcal{L}_{\mathrm{DPO}}(\pi_\theta; \pi_{\mathrm{ref}}) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\mathrm{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\mathrm{ref}}(y_l \mid x)}\right)\right]$$

Direct Preference Optimization: Analysis

Google Research

- **Loss:**

$$\mathcal{L}_{\mathrm{DPO}}(\pi_\theta; \pi_{\mathrm{ref}}) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\mathrm{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\mathrm{ref}}(y_l \mid x)}\right)\right]$$

- **What happens? (Gradient Analysis)**

$$\nabla_\theta \mathcal{L}_{\mathrm{DPO}}(\pi_\theta; \pi_{\mathrm{ref}}) = -\beta \mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\Bigg[\underbrace{\sigma\big(\hat{r}_\theta(x, y_l) - \hat{r}_\theta(x, y_w)\big)}_{\text{higher weight when reward estimate is wrong}}\Bigg[\underbrace{\nabla_\theta \log \pi(y_w \mid x)}_{\text{increase likelihood of } y_w} - \underbrace{\nabla_\theta \log \pi(y_l \mid x)}_{\text{decrease likelihood of } y_l}\Bigg]\Bigg]$$

where $\hat{r}_\theta(x, y) = \beta \log \frac{\pi_\theta(y|x)}{\pi_{\mathrm{ref}}(y|x)}$.

Direct Preference Optimization (DPO, Rafailov et al. 2024)

Google Research

- **Offline approach for learning from preferences**
  - Does not require sampling with a model
  - Can re-use SFT pipelines; much cheaper to run
- **No need to learn a reward/preferences model**
- **Issues:**
  - The preferences are measured with respect to the initial model or data
  - **Teacher-forcing / Off-policy:**
    - The signal does not evolve with the policy
    - The policy is frozen from the algorithm's perspective
  - Assumes regularization to the base model
  - Relies on the Bradley-Terry assumption

SimPO: Simple Preference Optimization (Meng et al., 2024)

Google Research

- **DPO without anchor regularization:**

$$\mathcal{L}_{\mathrm{SimPO}}(\pi_\theta) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\log\sigma\left(\frac{\beta}{|y_w|}\log\pi_\theta(y_w \mid x) - \frac{\beta}{|y_l|}\log\pi_\theta(y_l \mid x) - \gamma\right)\right]$$

- **Key changes:**
  - Length-normalized log-likelihood replaces reference model ratio
  - Replaces the anchor regularization with a fixed target reward margin $\gamma > 0$

Why Preference-Based RL?

Google Research

Vanilla RLHF: Learn a reward from preferences

- **However:**
  - Data is made of **preferences** and not scores!
  - Reward model can be misspecified (Bradley-Terry assumption)
  - Human preference is not always captured by a scalar reward

$$\max_{\pi} \min_{\pi'} \mathbb{E}\left[\mathcal{P}(y \succ y' \mid x) \,\middle|\, y \sim \pi,\, y' \sim \pi'\right]$$

Instead, learn directly from preferences:

- Find a policy which is preferred over any other policy (Nash Equilibrium).

Problems with Reward Model

Google Research

- **Cannot capture non-transitive preferences:**
  - $A \succ B \succ C \succ A$ (Rock-Paper-Scissors cycle)
- **Maximizing Elo $\neq$ maximizing win rate:**
  - Constrained setting: $\pi(y_1) = 2\pi(y_2)$
  - Elo-optimal policy: $\pi_R^* = (\frac{2}{3}, \frac{1}{3}, 0)$
  - Preference-optimal policy: $\pi_{\mathcal{P}}^* = (0, 0, 1)$ beats π_R^*:

$$\mathcal{P}(\pi_{\mathcal{P}}^* \succ \pi_R^*) = \mathcal{P}(y_3 \succ y_1) \cdot \tfrac{2}{3} + \mathcal{P}(y_3 \succ y_2) \cdot \tfrac{1}{3} = \tfrac{50}{99} > \tfrac{1}{2}$$

$\mathcal{P}(y \succ y')$	$y = y_1$	$y = y_2$	$y = y_3$
$y' = y_1$	1/2	9/10	2/3
$y' = y_2$	1/10	1/2	2/11
$y' = y_3$	1/3	9/11	1/2

Problems with Reward Model

Google Research

Setup: Cyclic Population Preferences

- **Groups:** A, B, C **Actions:** $\{1, 2, 3\}$
- **Near-uniform population:**
  - $P(A) = \frac{1}{3} - \varepsilon$
  - $P(B) = P(C) = \frac{1}{3} + \frac{\varepsilon}{2}$
- **Cyclic preferences** (0.5 otherwise):
  - Group A: prefers $2 \succ 1$
  - Group B: prefers $1 \succ 3$
  - Group C: prefers $3 \succ 2$

RLHF Collapse vs. Nash Equilibrium

- **Aggregate preference:**

$$P_\varepsilon = (\tfrac{1}{3} - \varepsilon)P_A + (\tfrac{1}{3} + \tfrac{\varepsilon}{2})(P_B + P_C)$$

- **Standard RLHF (Problem):**
  - Collapses deterministically to **Action 1** just because Group A is slightly smaller
- **Nash Equilibrium (Optimal):**
  - $\pi^* = (\frac{1}{3} + \frac{\varepsilon}{2},\ \frac{1}{3} + \frac{\varepsilon}{2},\ \frac{1}{3} - \varepsilon)$
  - Preserves **policy diversity** across groups!

Preference-Based RL

Google Research

- **Mathematical definition:**

$$\mathcal{P}(\pi \succ \pi') \stackrel{\text{def}}{=} \mathbb{E}_{x\sim\rho}\, \mathbb{E}_{y\sim\pi(\cdot|x),\, y'\sim\pi'(\cdot|x)} \left[\mathcal{P}(y \succ y' \mid x)\right]$$

- **Objective:**
  - Find a policy that is preferred over any other policy:

$$\pi^* \stackrel{\text{def}}{=} \arg\max_{\pi} \min_{\pi'} \mathcal{P}(\pi \succ \pi')$$

  - Min-max zero-sum game between two policies

Direct Preference Optimization: Beyond Bradley-Terry

Google Research

- **DPO relies on the reward assumption:**

$$\max_{\pi_\theta} \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi_\theta(y|x)} \left[r_\phi(x,y) \right] - \beta D_{\mathrm{KL}} \left[\pi_\theta(y \mid x) \,\|\, \pi_{\mathrm{ref}}(y \mid x) \right]$$

  - Vanilla RL maximizes rewards — so we needed to convert preferences to rewards

- **What if we instead operate directly on preferences? (ΨPO, Azar et al. 2023)**

$$\max_{\pi} \mathbb{E}_{\substack{x\sim\rho \\ y\sim\pi(\cdot|x) \\ y'\sim\mu(\cdot|x)}} \left[\Psi(p^*(y \succ y' \mid x)) \right] - \tau D_{\mathrm{KL}}(\pi \,\|\, \pi_{\mathrm{ref}})$$

  - Can be shown as a more general case than DPO ($\Psi(q) = \log \frac{q}{1-q}$)

- **Identity Preference Optimization (IPO):** directly optimize preferences ($\Psi(q) = q$):

$$\max_{\pi} p^*_\rho(\pi \succ \mu) - \tau D_{\mathrm{KL}}(\pi \,\|\, \pi_{\mathrm{ref}})$$

Online Preference Optimization & Nash-MD

Google Research

- **From Offline to Online Preferences** *(Calandriello et al., 2024; Guo et al., 2024)*:
  - Offline DPO/IPO suffers from distribution shift as π_θ drifts away from the static dataset
  - **Online fix:** Sample $y, y' \sim \pi_\theta(\cdot \mid x)$ on-policy and score with a preference model $\mathcal{P}(y \succ y' \mid x)$
- **Nash Learning from Human Feedback (Nash-MD)** *(Munos et al., 2024)*:
  - Solve the KL-regularized minimax game (staying close to anchor μ):

$$\max_{\pi} \min_{\pi'} \mathcal{P}(\pi \succ \pi') - \alpha D_{\mathrm{KL}}(\pi \,\|\, \mu) + \alpha D_{\mathrm{KL}}(\pi' \,\|\, \mu)$$

  - **Key Idea (Self-Play Mirror Descent):** Improve π_{t+1} against π_t while staying close to μ and π_t:

$$\pi_{t+1}(\cdot \mid x) = \arg\max_{\pi} \; \eta_t \mathcal{P}(\pi \succ \pi_t \mid x) - \alpha \eta_t D_{\mathrm{KL}}(\pi \,\|\, \mu)[x] - (1 - \alpha\eta_t) D_{\mathrm{KL}}(\pi \,\|\, \pi_t)[x]$$

  - Converges to the unique Nash equilibrium — **we will cover the full algorithm & analysis in Multi-turn RL!**

RL with Verifiable Rewards

Rewards

Google Research

- **Human feedback:**
  - Heuristic on Goldens
  - Preferences (alignment)
  - Thumbs-up / Thumbs-down
- **Verifiable:**
  - Math / Coding / Information Seeking

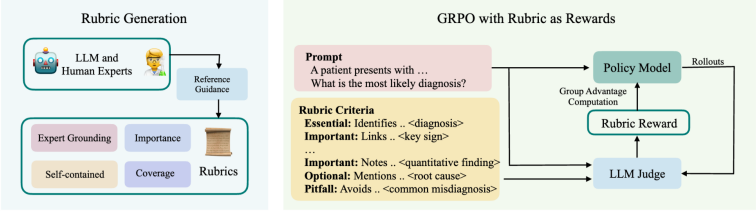
Rewards

Google Research

- Human feedback is used for soft-tasks where success is hard to capture
- Verifiable tasks allow for a more consistent reward (math/coding)

Can we have verifiable rewards for soft-tasks?

- **Critique-Based (LLM-as-a-Judge):**
  - **Generative:** Judge produces a Chain-of-Thought critique before scoring to improve accuracy
  - **Rubrics:** Decompose open-ended quality into explicit, verifiable checklist criteria



RLVR & Emergent Reasoning (OpenAI o1, 2024; DeepSeek-R1, 2025)

Google Research

- **Pure RL on Verifiable Rewards (DeepSeek-R1-Zero):**
  - Train directly with RL using binary rule-based rewards (accuracy + format), without PRM or SFT
  - Thinking time (CoT length) naturally scales up during training as the model tackles harder problems
- **Emergent Test-Time Reasoning Behaviors:**
  - Spontaneous **self-verification**, **backtracking**, and **reflection** (*"Wait, let me re-evaluate. . . "*)
  - Shifts compute from model parameters to **test-time thinking tokens**
- **Practical Multi-Stage Recipe (DeepSeek-R1)** — fixes R1-Zero's readability & language mixing:
  1. **Cold-start SFT:** Fine-tune on a small set of curated, readable long-CoT examples
  2. **Reasoning RL:** RL on verifiable tasks (math, code, logic) + language consistency reward
  3. **Rejection Sampling SFT:** Filter correct RL rollouts and combine with general SFT data
  4. **Hybrid RL:** Final RL stage combining verifiable rewards (reasoning) and RLHF (general tasks)

Regularization Revisited

Google Research

- **Is anchor regularization still needed?**
  - Anchor can be seen as a way not to diverge from the original model
  - Some variants remove anchor regularization (e.g., **SimPO**, **DAPO**)
- **This is an open discussion:**
  - **Preference vs. Verifiable:** Verifiable rewards are harder to hack than learned reward models
  - If using PPO / trust-region / step-wise KL regularization, the updates are still proximal

Credit Assignment in LLMs

Auto-regressive Models and RL

Google Research

- **So far we learned about RLHF — Is it truly RL?**
- **Single-turn:**
  - Model generates a sequence of actions; no external environment
  - **Actually a contextual bandit problem!**
  - The true action is the whole generation; autoregression is just a modeling choice
  - However, language is indeed causal, and CoT can be seen as compute
- **Credit assignment is still an issue:**
  - How to attribute the reward to different parts of the sequence?
  - Especially important when Chain-of-Thought (CoT) is used.

Value Baseline in LLMs

Google Research

- **Value is fundamental in RL:**
  - Handles credit assignment
  - Advantage reduces variance (REINFORCE with baseline)
- **Learning value in LLMs:**
  - A language model outputs tokens
  - Can remove the unembedding head and create a new scalar value head following the representation
- **Problems:**
  - No value pretraining
  - Hard to train accurately at LLM scale

Outcome vs. Process Reward Models (ORMs vs. PRMs)

Google Research

- **Outcome Reward Models (ORMs):**
  - Score only the final response $R(x, y) \implies$ sparse credit assignment over long Chain-of-Thought
  - Model can reach the right final answer via flawed intermediate reasoning (false positives)
- **Process Reward Models (PRMs)** *(Lightman et al., 2023; Wang et al., 2024)*:
  - Score each intermediate reasoning step $r(x, y_{\leq k})$ to provide dense step-level credit
  - Trained via human step annotations or automated Monte-Carlo rollouts from step k
- **Why PRMs Are Challenging in Large-Scale RL:**
  - Expensive to label (requires many rollouts per step) and hard to define step boundaries
  - Vulnerable to **step-level reward hacking** (e.g., generating trivial or repetitive steps)

Single Prompt, Multiple Rollouts

Google Research

- **Academic RL:**
  - Overfitting: see only a single domain many times until convergence
- **RL for LLMs (Meta-RL):**
  - Generalization: We want to be good at unseen problems (prompts)
- **Practical solution: Sample N candidate solutions for each prompt**
  - We see more than a single solution
  - Solutions are generated in parallel — no environment state transition across rollouts
  - **Comparison is important:** understanding what is good builds on comparing to alternatives
  - Enables shallow exploration to find and teach the policy to differentiate between good and bad solutions
- **Should we see a prompt more than once?**
  - Naively, solving complex problems requires many repetitions
  - Practically, generalization allows working once (or few times) on each prompt

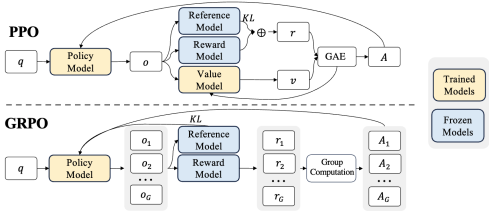
GRPO: Group Relative Policy Optimization (Shao et al., 2024)

Google Research

- Sample G generations per prompt
- Monte-Carlo estimate of the advantage (normalized by std):

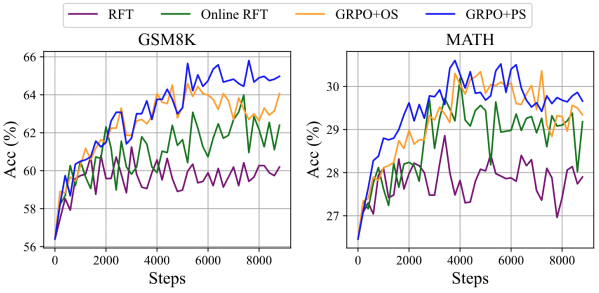
$$A_i = \frac{r_i - \text{mean}\big(\{r_1, r_2, \cdots, r_G\}\big)}{\text{std}\big(\{r_1, r_2, \cdots, r_G\}\big)}$$

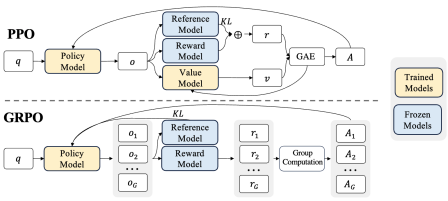
- **Comments:**
  - Baseline is estimated just at the prompt level
  - Credit assignment for later tokens is not truly addressed
  - In RLHF, std-normalizing helps in scaling the advantage
  - **Zero-variance prompts do not get any signal!**



GRPO: Empirical Performance (Shao et al., 2024)

Google Research





Beyond Vanilla RL

Naive RL: Recursive Improvement (ReST)

Google Research

- See *Reinforced Self-Training (ReST) for Language Modeling* (Gulcehre et al., 2023)

A repetitive two-phase algorithm:

- **Grow:**
  - Sample data out of the current policy
  - Possible to sample N generations per prompt, or just a single one
- **Improve:**
  - Filter data according to reward signal and threshold τ:

$$F(x, y; \tau) = \mathbb{I}_{R(x,y)>\tau}$$

  - Use a loss for training on the selected examples:

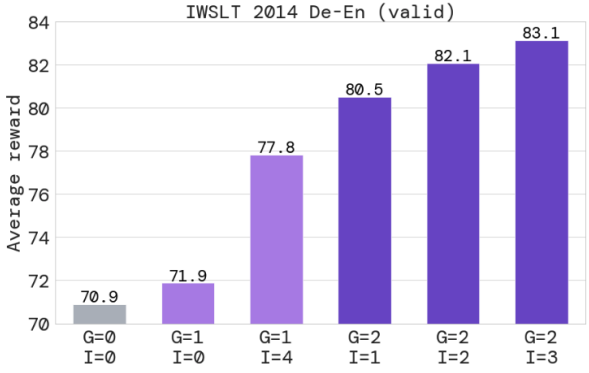
$$J(\theta) = \mathbb{E}_{(x,y)\sim\mathcal{D}_g}\left[F(x, y; \tau)\,\mathcal{L}(x, y; \theta)\right]$$

  - Usually just SFT (negative log-likelihood) on the filtered examples

Naive RL: Recursive Improvement (Results)

Google Research

- *Reinforced Self-Training (ReST) for Language Modeling* (2023) — iterative Grow & Improve stages:



DAPO: Tweaking the RL Pipeline and Algorithm (1/4) (Yu et al., 2025)

Google Research

$$\mathcal{J}_{\text{DAPO}}(\theta) = \mathbb{E}_{(q,a)\sim\mathcal{D},\ \{o_i\}_{i=1}^{G}\sim\pi_{\theta_{\text{old}}}(\cdot|q)}\left[\frac{1}{\sum_{i=1}^{G}|o_i|}\sum_{i=1}^{G}\sum_{t=1}^{|o_i|}\min\Big(r_{i,t}(\theta)\hat{A}_{i,t},\ \text{clip}\Big(r_{i,t}(\theta), 1-\varepsilon_{\text{low}}, 1+\varepsilon_{\text{high}}\Big)\hat{A}_{i,t}\Big)\right]$$

$$\text{s.t.}\quad 0 < \Big|\{o_i \mid \text{is_equivalent}(a, o_i)\}\Big| < G, \qquad r_{i,t}(\theta) = \frac{\pi_\theta(o_{i,t} \mid q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} \mid q, o_{i,<t})}, \quad \hat{A}_{i,t} = \frac{R_i - \text{mean}(\{R_i\}_{i=1}^{G})}{\text{std}(\{R_i\}_{i=1}^{G})}$$

- **Remove anchor regularization (no KL penalty):**
  - On verifiable tasks, KL regularization to π_{ref} is not necessarily needed, as the reward is grounded and verifiable.

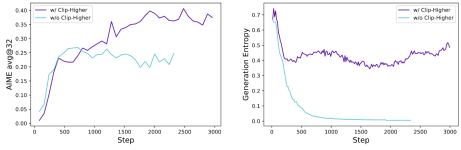
DAPO: Tweaking the RL Pipeline and Algorithm (2/4) (Yu et al., 2025)

Google Research

$$\mathcal{J}_{\text{DAPO}}(\theta) = \mathbb{E}_{(q,a)\sim\mathcal{D},\ \{o_i\}_{i=1}^{G}\sim\pi_{\theta_{\text{old}}}(\cdot|q)}\left[\frac{1}{\sum_{i=1}^{G}|o_i|}\sum_{i=1}^{G}\sum_{t=1}^{|o_i|}\min\Big(r_{i,t}(\theta)\hat{A}_{i,t},\ \text{clip}\Big(r_{i,t}(\theta),\ \boxed{1-\varepsilon_{\text{low}},\ 1+\varepsilon_{\text{high}}}\Big)\hat{A}_{i,t}\Big)\right]$$

$$\text{s.t.}\quad 0 < \Big|\{o_i \mid \text{is_equivalent}(a, o_i)\}\Big| < G, \qquad r_{i,t}(\theta) = \frac{\pi_\theta(o_{i,t} \mid q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} \mid q, o_{i,<t})}, \quad \hat{A}_{i,t} = \frac{R_i - \text{mean}(\{R_i\}_{i=1}^{G})}{\text{std}(\{R_i\}_{i=1}^{G})}$$

- **Asymmetric clipping "CLIP-HIGHER":**
  - Vanilla PPO uses a single clipping threshold ε
  - This implies that tokens with higher probability are less constrained
  - For tokens with small probability, increasing the probability becomes challenging (entropy collapse)



(a) Accuracies on AIME. **(b)** Entropy of actor model.

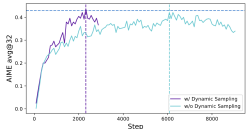
DAPO: Tweaking the RL Pipeline and Algorithm (3/4) (Yu et al., 2025)

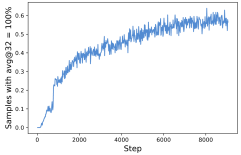
Google Research

$$\mathcal{J}_{\text{DAPO}}(\theta) = \mathbb{E}_{(q,a)\sim\mathcal{D},\ \{o_i\}_{i=1}^{G}\sim\pi_{\theta_{\text{old}}}(\cdot|q)}\left[\frac{1}{\sum_{i=1}^{G}|o_i|}\sum_{i=1}^{G}\sum_{t=1}^{|o_i|}\min\Big(r_{i,t}(\theta)\hat{A}_{i,t},\ \text{clip}\Big(r_{i,t}(\theta), 1-\varepsilon_{\text{low}}, 1+\varepsilon_{\text{high}}\Big)\hat{A}_{i,t}\Big)\right]$$

$$\text{s.t.}\quad 0 < \Big|\{o_i \mid \text{is_equivalent}(a, o_i)\}\Big| < G, \qquad r_{i,t}(\theta) = \frac{\pi_\theta(o_{i,t} \mid q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} \mid q, o_{i,<t})}, \qquad \hat{A}_{i,t} = \frac{R_i - \text{mean}(\{R_i\}_{i=1}^{G})}{\text{std}(\{R_i\}_{i=1}^{G})}$$

- **Dynamic Sampling:**
  - With GRPO, prompt accuracy 0 or 1 means zero advantage
  - This issue increases during training
  - These samples should be removed for training stability, so all batches have effective gradients





(b) The proportion of samples with an accuracy of 1.

DAPO: Tweaking the RL Pipeline and Algorithm (4/4) (Yu et al., 2025)

Google Research

$$\mathcal{J}_{\mathrm{DAPO}}(\theta) = \mathbb{E}_{(q,a)\sim\mathcal{D},\ \{o_i\}_{i=1}^{G}\sim\pi_{\theta_{\mathrm{old}}}(\cdot|q)} \left[\frac{1}{\sum_{i=1}^{G}|o_i|} \sum_{i=1}^{G}\sum_{t=1}^{|o_i|} \min\left(r_{i,t}(\theta)\hat{A}_{i,t},\ \mathrm{clip}\left(r_{i,t}(\theta), 1-\varepsilon_{\mathrm{low}}, 1+\varepsilon_{\mathrm{high}}\right)\hat{A}_{i,t} \right) \right]$$

$$\text{s.t.}\quad 0 < \left| \{ o_i \mid \mathrm{is_equivalent}(a, o_i) \} \right| < G, \qquad r_{i,t}(\theta) = \frac{\pi_\theta(o_{i,t} \mid q, o_{i,<t})}{\pi_{\theta_{\mathrm{old}}}(o_{i,t} \mid q, o_{i,<t})}, \quad \hat{A}_{i,t} = \frac{R_i - \mathrm{mean}(\{R_i\}_{i=1}^{G})}{\mathrm{std}(\{R_i\}_{i=1}^{G})}$$

- **Token-wise loss:**
  - Normalize across total generated tokens $\sum_{i=1}^{G}|o_i|$ (rather than averaging per response first)

Training vs. Inference

Google Research

A task with verifiable binary rewards

- **In GRPO:**
  - **Generation:** *k* rollouts for Monte-Carlo value estimation
  - **Optimization:** optimizing $\mathrm{pass@1}$ (regular RL)
- Optimizing $\mathrm{pass@}k$ would help to improve exploration & training
- Optimizing $\mathrm{pass@1}$ is our actual goal for single-sample inference

MaxRL: Maximum Likelihood RL (Tajwar et al., 2026)

Google Research

- **What if we try to optimize success as in SFT (Maximum Likelihood)?**

$$\nabla_\theta J_{\mathrm{RL}} = \mathbb{E}_{x\sim\rho}\big[\nabla_\theta p_\theta(x)\big]$$
$$\nabla_\theta J_{\mathrm{ML}} = \mathbb{E}_{x\sim\rho}\big[\nabla_\theta \log p_\theta(x)\big] = \mathbb{E}_{x\sim\rho}\left[\frac{1}{p_\theta(x)}\nabla_\theta p_\theta(x)\right]$$

- $p_\theta(x)$ denotes the **probability of success** on prompt x (not the policy itself!)
- You can think of the policy $m_\theta(z \mid x)$ as a latent variable generator:

$$p_\theta(y \mid x) = \sum_{z\in\mathcal{Z}} m_\theta(z \mid x)\,\mathbb{I}\{f(z) = y\}$$

MaxRL: Taylor Expansion & Estimator

Google Research

- **Let's do a Taylor expansion:**

$$J_{\mathrm{ML}}(x) = \log p = -\sum_{k=1}^{\infty} \frac{(1-p)^k}{k} = -\sum_{k=1}^{\infty} \frac{\mathrm{fail@}k(x)}{k}$$

- **Optimization tries to maximize all $\mathrm{pass@}k$:**

$$\nabla_\theta J_{\mathrm{ML}}(x) = \sum_{k=1}^{\infty} \frac{1}{k} \nabla_\theta \mathrm{pass@}k(x)$$

  - Interpolates between training ($\mathrm{pass@}k$) and test ($\mathrm{pass@}1$)
- **Replacement for GRPO:**
  - Instead of dividing by the STD, divide by $K = N\hat{r}(x)$, the number of successes
  - The rarer the success, the higher the gradient weight:

$$\hat{g}(x) \leftarrow \begin{cases} \dfrac{1}{N\hat{r}(x)} \displaystyle\sum_{j=1}^{N} \big(r_j - \hat{r}(x)\big) S_j, & \hat{r}(x) > 0, \\ 0, & \text{otherwise} \end{cases} \qquad \text{where } S_i := \nabla_\theta \log m_\theta(z_i \mid x)$$

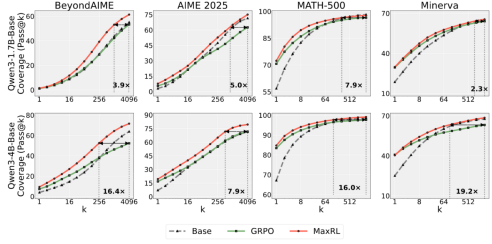
MaxRL: Scaling Difference & Results

Google Research

- **The prompt-level gradient weight scaling difference:**

	RL	GRPO	MaxRL (T)	ML
$w(p)$	1	$\frac{1}{\sqrt{p(1-p)}}$	$\frac{1-(1-p)^T}{p}$	$\frac{1}{p}$

- GRPO is better than baseline on $\mathrm{pass@1}$, but lower on $\mathrm{pass@}N$ for big N
- MaxRL is always superior to the baseline as it jointly optimizes $\mathrm{pass@}k$ for all k



Test-Time Compute & Inference-Time Scaling (Snell et al., 2024)

Google Research

- **Trading Inference Compute for Performance:**
  - Allocate more test-time FLOPs on hard prompts instead of only scaling model parameters
- **Two Axes of Test-Time Scaling:**
  - **Parallel Sampling ($\text{pass@}k \rightarrow \text{pass@}1$):** Best-of-*N* with a verifier/reward model, or Majority Voting
  - **Sequential Revision & Search:** Long internal CoT trace, iterative refinement, or tree search (MCTS)
- **Closing the Loop — Distilling Test-Time Compute Back into the Policy:**
  - Use expensive inference-time search / Best-of-*N* as a **teacher** to improve single-sample π_θ
  - **Best-of-*N* Distillation (BOND)** *(Sessa et al., 2024)*: Train π_θ to match the Best-of-*N* distribution on-policy

On-Policy Distillation: Teacher vs. Student

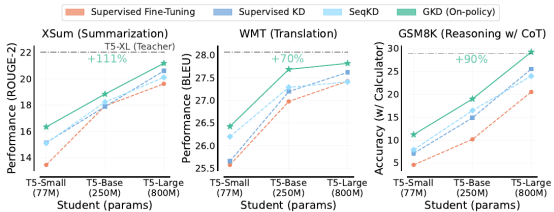
Google Research

- **Teacher–Student:** A larger **teacher** p_{T} trains a smaller, faster **student** p_{S}^{θ}
- **Supervised Distillation (Teacher Forcing):**
  - Imitate teacher on fixed data $\Longrightarrow$ **train–inference mismatch**

$$\mathcal{L}_{\mathrm{SD}}(\theta) = \mathbb{E}_{(x,y)\sim(X,Y)}\left[D_{\mathrm{KL}}(p_{\mathrm{T}} \,\|\, p_{\mathrm{S}}^{\theta})(y \mid x)\right]$$

- **On-Policy Distillation** *(GKD, Agarwal et al., 2024)*:
  - Student generates its own rollouts and gets teacher feedback on its mistakes

$$\mathcal{L}_{\mathrm{OD}}(\theta) = \mathbb{E}_{x\sim X,\, y\sim p_{\mathrm{S}}^{\theta}(\cdot|x)}\left[\mathcal{D}(p_{\mathrm{T}} \,\|\, p_{\mathrm{S}}^{\theta})(y \mid x)\right]$$



Multi-turn & Agentic RL

The LLM Evolution: From Answers to Agents

Google Research

Three Generations of LLM Capabilities:

1. **Single-Turn Generation:** Directly map a prompt to a final answer in a single pass
2. **Internal Reasoning (Chain-of-Thought):** Adapt compute via thinking tokens — an *"inner tool"*
3. **Agentic & Multi-Turn Interaction:** Agents communicate and act in the world

Comment:

- User interaction was always multi-turn in practice (recovering from errors, follow-up requests)
- Yet LLMs were historically trained only for single-turn responses!

What Is Agentic RL? Interacting with the World

Google Research

Three Ways to Interact with the External World:

1. **Humans:** Communicating with a user through multi-turn dialogue
2. **Tools:** Editing code, calling search, and running external programs
3. **Other Agents:** Delegating tasks or collaborating in multi-agent systems

Deterministic vs. Stochastic Environment Transitions:

- **Tools** (compilers, calculators, static lookups) are typically **deterministic**
- **Humans** are inherently **stochastic** (hidden state, evolving intent, diverse responses)
- **Other Agents:** Using another LLM agent as a tool makes the transition **stochastic** too!

Motivation: Trajectory-Level Optimization

Google Research

- **Communication and tool-use are multi-turn processes**
  - Both user conversations and agentic workflows unfold over several dependent turns
  - Achieving complex goals requires planning across multiple interactions!
- **Myopic (turn-level) feedback is insufficient**
  - Optimizing single-turn preference $\neq$ good long-term trajectory behavior
  - E.g., asking a clarifying question or running a test sacrifices immediate completion for final success
- **Core Principle:** RL must optimize **conversation** / **trajectory-level** outcomes

Conversation Level Feedback

Google Research

Which is better?

User: Can you give me some recommendations for dinner in New York City for less than $50?

Agent A

Agent: Sure, here are some recommended spots in New York City. [. . .]

User: I was looking more for Mexican or Southwestern spots.

Agent: In that case, how about these Mexican or Southwestern spots? [. . .]

Agent B

Agent: Sure, any particular options you had in mind?

User: Mostly some Mexican or Southwestern spots.

Agent: Perfect, how about these Mexican or Southwestern spots? [. . .]

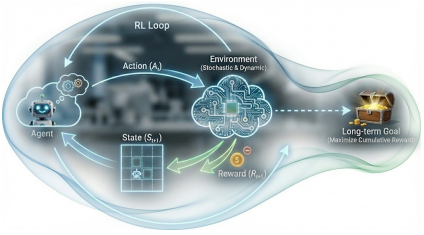
- **Easier to tell by the end of the conversation!**

Multi-turn vs. Single-turn

Google Research

Multi-turn is an RL scenario:

- Compare two episodes (conversations)
- Environment (user / tool-use) turn between agent turns
- Preference is over the **whole conversation**



Multi-turn Reinforcement Learning from Preference Human Feedback

Google Research

RL method for learning from conversation-level preferences (Shani et al., 2024):

- **Theory:** Provide convergence guarantees
- **Practice:** Implement the algorithm and show its benefits



Multi-turn Preference Optimization (MTPO)

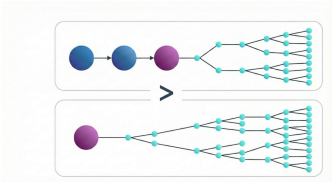
Google Research

- **Objective:** Find a policy that is preferred over any other policy
- **Self-Play:** Improve the policy against itself
- **Policy Gradient:** How much in expectation:

the policy from the current state

$>$

the policy from the start state



- **Update:** Mirror Descent Policy Optimization on self-play policy gradient
  - Converges to a unique Nash Equilibrium

MTPO: Formulation

Google Research

- Like the single-turn case, we want to keep proximity to the anchor μ:

$$\mathcal{P}_\alpha(\pi \succ \pi' \mid c) \triangleq \mathcal{P}(\pi \succ \pi' \mid c) - \alpha \mathrm{KL}_p(\pi \,\|\, \mu) + \alpha \mathrm{KL}_p(\pi' \,\|\, \mu)$$

- **Objective:**

$$\max_{\pi} \min_{\pi'} \mathbb{E}_p\left[\mathcal{P}_\alpha(\pi \succ \pi' \mid c)\right]$$

  - Find a policy that is preferred over any other policy, given that the policies are not too far from the initial model μ.

MTPO: Policy Gradient

Google Research

- We derive the **policy gradient** for the preference objective:

$$\nabla_{\pi(y_h|x_h)} \mathcal{P}(\pi \succ \pi') = \Pr(x_h \mid \pi) \underbrace{\sum_{x_{H+1}} \Pr(x_{H+1} \mid x_h, y_h, \pi)\, \mathcal{P}(x_{H+1} \succ \pi')}_{Q^{\pi,\pi'}(x_h, y_h)}$$

- **Regularized Q-function:**

$$Q_\alpha^{\pi,\pi'}(x_h, y_h) = \mathbb{E}_{\pi,p}\left[\mathcal{P}(x_{H+1} \succ \pi') - \alpha \sum_{h'=h}^{H} \mathrm{KL}(\pi \,\|\, \mu)[x_{h'}] \,\middle|\, x_h, y_h \right]$$

  - Expected preference of following policy π from (x_h, y_h) over following policy π' from the start state

- A new term — does not appear in the single-turn case!
- Can be learned similarly to a regular Q-function



MTPO: Algorithm

Google Research

Policy update:

$$\pi_{t+1}(\cdot \mid x_h) = \arg\max_{\pi} \ \eta_t \left\langle \pi, Q_\alpha^{\pi_t, \pi_t} \right\rangle [x_h] - \alpha\eta_t \mathrm{KL}(\pi \,\|\, \mu)[x_h] - (1 - \alpha\eta_t)\mathrm{KL}(\pi \,\|\, \pi_t)[x_h]$$

- **Regularized self-play mirror descent update:**
  - Improve an action if it is preferred over the current policy in expectation
  - Regularization: Stay close to the anchor μ
  - MD-term: Stay close to the previous policy π_t

MTPO: Algorithm

Google Research

Policy update:

$$\pi_{t+1}(\cdot \mid x_h) = \arg\max_{\pi} \boxed{\eta_t \left\langle \pi, Q_\alpha^{\pi_t, \pi_t} \right\rangle [x_h]} - \alpha\eta_t \mathrm{KL}(\pi \,\|\, \mu)[x_h] - (1 - \alpha\eta_t)\mathrm{KL}(\pi \,\|\, \pi_t)[x_h]$$

- **Regularized self-play mirror descent update:**
  - **Improve an action if it is preferred over the current policy in expectation**
  - Regularization: Stay close to the anchor μ
  - MD-term: Stay close to the previous policy π_t

MTPO: Algorithm

Google Research

Policy update:

$$\pi_{t+1}(\cdot \mid x_h) = \arg\max_{\pi} \ \eta_t \left\langle \pi, Q_\alpha^{\pi_t, \pi_t} \right\rangle [x_h] \boxed{-\alpha\eta_t \mathrm{KL}(\pi \,\|\, \mu)[x_h]} - (1 - \alpha\eta_t)\mathrm{KL}(\pi \,\|\, \pi_t)[x_h]$$

- **Regularized self-play mirror descent update:**
  - Improve an action if it is preferred over the current policy in expectation
  - **Regularization: Stay close to the anchor μ**
  - MD-term: Stay close to the previous policy π_t

MTPO: Algorithm

Google Research

Policy update:

$$\pi_{t+1}(\cdot \mid x_h) = \arg\max_{\pi} \ \eta_t \left\langle \pi, Q_\alpha^{\pi_t, \pi_t} \right\rangle [x_h] - \alpha\eta_t \mathrm{KL}(\pi \,\|\, \mu)[x_h] \boxed{-(1-\alpha\eta_t)\mathrm{KL}(\pi \,\|\, \pi_t)[x_h]}$$

- **Regularized self-play mirror descent update:**
  - Improve an action if it is preferred over the current policy in expectation
  - Regularization: Stay close to the anchor μ
  - **MD-term: Stay close to the previous policy π_t**

MTPO: Algorithm

Google Research

Policy update:

$$\pi_{t+1}(\cdot \mid x_h) = \arg\max_{\pi} \ \eta_t \left\langle \pi, Q_\alpha^{\pi_t, \pi_t} \right\rangle [x_h] - \alpha\eta_t \mathrm{KL}(\pi \,\|\, \mu)[x_h] - (1 - \alpha\eta_t)\mathrm{KL}(\pi \,\|\, \pi_t)[x_h]$$

- **Regularized self-play mirror descent update:**
  - Improve an action if it is preferred over the current policy in expectation
  - Regularization: Stay close to the anchor μ
  - MD-term: Stay close to the previous policy π_t
- **State-level update!** (does not optimize over the entire trajectory at once)

MTPO: Convergence

Google Research

Theorem 4.2 (Convergence to Unique Nash Equilibrium)

Let π_α^* be the Nash equilibrium of the regularized preference model, and $\mathbb{Q}$ be a bound on the magnitude of the Q-functions. Then, for $\eta_t = \frac{2}{\alpha(t+2)}$, MTPO guarantees at every iteration t:

$$\mathrm{KL}_p(\pi_\alpha^* \,\|\, \pi_t) \leq \frac{32H\mathbb{Q}^2}{\alpha^2(t+1)}.$$

Let $\mu_{\min}$ be the minimal non-zero probability assigned by μ, then $\mathbb{Q} \leq \max\left\{4\alpha H \log \frac{1}{\mu_{\min}},\, 1\right\}$.



MTPO: Convergence (Proof Sketch)

Google Research

Proof Sketch:

- **Two policies = Two Players:** A zero-sum game analysis
- **Self-play:** Due to the symmetry of the players
- **Adversarial MDPs:** Each player can view the other player as an adversarial choice of rewards chosen prior to each iteration
- **Uniqueness:** Anchor regularization makes the problem strongly convex, allowing convergence to a unique policy π_α^*

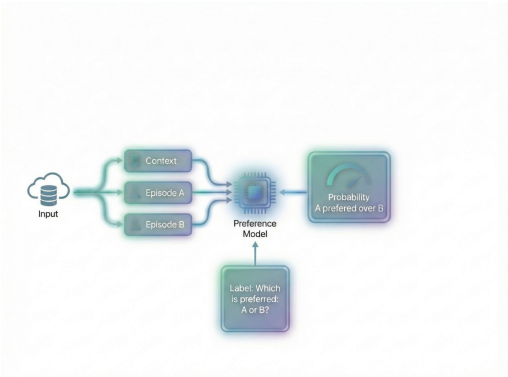


MTPO: Implementation — Preference Model

Google Research

Preference Model Learning

- **Input:** Context, Episode A, Episode B
- **Label:** Which is preferred: A or B?
- **Output:** Preference probability $\mathcal{P}(\text{A} \succ \text{B} \mid c)$



MTPO: Implementation — Sampling & Actor-Critic

Google Research

Sampling

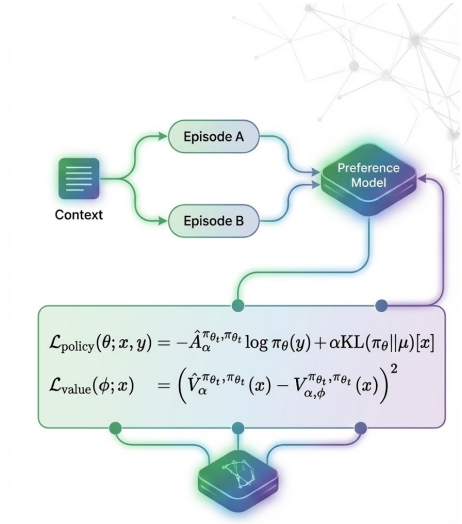
- Context is sampled from the data
- Two episodes are generated given the context
- Compare using the learned preference model

Learning

- Actor-critic policy gradient algorithm based on the new Q-function:

$$\mathcal{L}_{\text{policy}}(\theta; x, y) = -\hat{A}_{\alpha}^{\pi_{\theta_t}, \pi_{\theta_t}}(x, y) \log \pi_\theta(y \mid x) + \alpha \mathrm{KL}(\pi_\theta \,\|\, \mu)[x]$$

$$\mathcal{L}_{\text{value}}(\phi; x) = \left(\hat{V}_{\alpha}^{\pi_{\theta_t}, \pi_{\theta_t}}(x) - V_{\alpha,\phi}^{\pi_{\theta_t}, \pi_{\theta_t}}(x) \right)^2$$



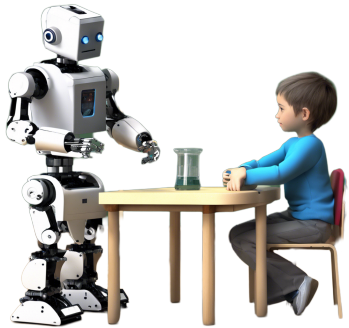
Experiments: Education Dialogue

Google Research

Scenario: The agent is required to teach the user some topic

Data:

- List of topics (Periodic table, Pythagorean theorem, etc.)
- **Gemini** is prompted to create conversations (per topic)
- **Gemini** judges between each two conversations



Experiments: Baselines

Google Research

Single-turn Reward



- Prompt that predicts the turn's effect on the entire conversation
- Simulates the way humans reason

Single-turn Value



- Single-turn preferences are estimated by unrolling the entire conversation

Multi-turn RLHF



- A conversation-level reward model learned using Bradley-Terry

Results: Education Dialogue

Google Research

Multi-turn significantly outperforms single-turn:

- Hard to capture the effect of a single decision on a conversation
- Depends on the rest of the conversation
- The policy changes in training, making static estimation biased

Multi-turn preference learning is better than multi-turn RLHF

	SL	**Single-turn-reward**		**Single-turn-value**		**Multi-turn**		
	SFT	RLHF	Nash	RLHF	Nash	RLHF	MTPO	MTPO-τ
SFT	–	0.164	0.347	0.197	0.324	0.212	0.091	0.093
RLHF-reward	0.836	–	0.628	0.515	0.654	0.399	0.392	0.354
Nash-reward	0.653	0.372	–	0.411	0.510	0.328	0.281	0.242
RLHF-value	0.803	0.485	0.589	–	0.568	0.408	0.396	0.366
Nash-value	0.676	0.346	0.490	0.432	–	0.450	0.298	0.270
RLHF-multi	0.788	0.601	0.672	0.592	0.550	–	0.433	0.412
MTPO	0.909	0.608	0.719	0.604	0.702	0.567	–	0.439
MTPO-τ	**0.907**	**0.646**	**0.758**	**0.634**	**0.730**	**0.588**	**0.561**	–

Summary: Multi-turn Preference Optimization

Google Research

- **We devise a preference-based multi-turn RL algorithm (MTPO)**
  - **Theory:** MTPO converges to the optimal Nash policy
  - **Practice:** MTPO outperforms multi-turn and single-turn baselines
- **Takeaway:** Learning from multi-turn feedback is crucial to unlock conversational & agentic LLMs

Multi-turn and Credit Assignment (1/2)

Google Research

- **In multi-turn GRPO, we generate *N* episodes**
  - This allows assigning credit at the episode level
  - Is it sufficient?
- **Example:**
  - **Episode A:**
    - **User Turn 1:** Write a pipeline in Python for training a model to play ATARI
    - **Agent Turn 1:** Code with bugs
    - **User Turn 2:** But it doesn't work, you have tons of bugs
    - **Agent Turn 2:** Working code
  - **Episode B:**
    - **User Turn 1:** Write a pipeline in Python for training a model to play ATARI
    - **Agent Turn 1:** Working code
- **Naively, GRPO will assign a good reward to the buggy code!**

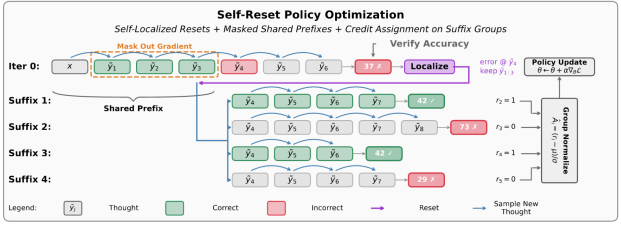
Multi-turn and Credit Assignment (2/2)

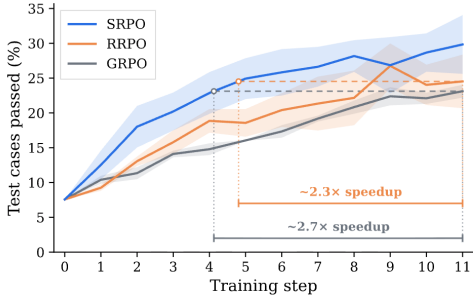
Google Research

Alternatives to vanilla GRPO:

- **Value function** — hard to train, but the didactic way
- **Do GRPO from an intermediate state along the episode**
  - **Splitting strategies:**
    - Uniform
    - At the first source of error — requires a post-hoc judge

Credit Assignment with Resets in Language Models (Samanta et al., 2026)





Multi-turn and DPO

Google Research

- **A multi-turn trajectory $\tau = (x_1, y_1, \ldots, x_H, y_H)$ includes environment transitions p:**

$$\Pr(\tau \mid \pi) = \rho(x_1) \prod_{h=1}^{H} \pi(y_h \mid x_h) \underbrace{p(x_{h+1} \mid x_h, y_h)}_{\text{environment / user}}$$

- **By soft Bellman optimality, the policy at turn h depends on the value function V^*:**

$$\beta \log \frac{\pi^*(y_h \mid x_h)}{\pi_{\text{ref}}(y_h \mid x_h)} = Q^*(x_h, y_h) - V^*(x_h) = r(x_h, y_h) + \mathbb{E}_{x'_{h+1} \sim p(\cdot \mid x_h, y_h)}\left[V^*(x'_{h+1})\right] - V^*(x_h)$$

- **Summing over the trajectory τ, intermediate values do not cancel in stochastic MDPs:**

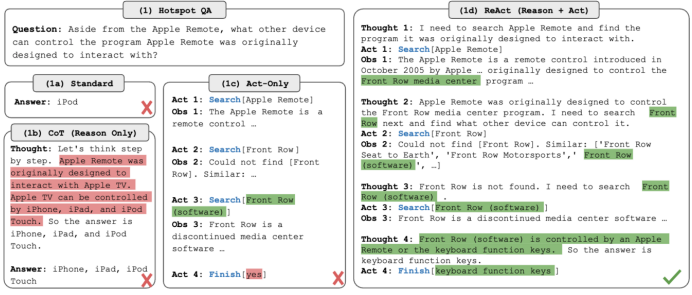
$$R(\tau) - V^*(x_1) = \sum_{h=1}^{H} \left[\beta \log \frac{\pi^*(y_h \mid x_h)}{\pi_{\text{ref}}(y_h \mid x_h)} + V^*(x_{h+1}) - \mathbb{E}_{x'_{h+1} \sim p(\cdot \mid x_h, y_h)}\left[V^*(x'_{h+1})\right] \right]$$

  - **Stochastic environment:** Requires learning a value function V^* (otherwise DPO conflates lucky environment transitions with good policy actions).
  - **Deterministic environment ($p(x_{h+1} \mid x_h, y_h) = 1$, e.g., single-turn token generation):** Environment probability & value discrepancy terms cancel out ($\mathbb{E}_p[V^*(x'_{h+1})] = V^*(x_{h+1})$), recovering standard DPO!

In-Context RL: ReAct and Reflection (1/2)

Google Research

- **ReAct** *(Yao et al., 2023)*: **Prompt an agentic model to act as a planning policy**
- **At each step:**
  - Model gets **Observation** (external knowledge)
  - Model **Thinks** / **Plans** (CoT)
  - Model **Acts**
- Creates a repetitive loop until the goal is achieved



In-Context RL: ReAct and Reflection (2/2)

Google Research

- **Reflection** *(Reflexion, Shinn et al., 2023)*: **Give feedback in context alongside the observation**
- **Feedback options:**
  - Another critique LLM that reflects on the action and result
  - Real external feedback — e.g., run unit tests / compute reward and include in context

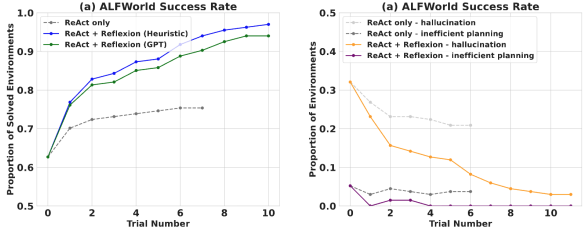


Figure 3: (a) AlfWorld performance across 134 tasks showing cumulative proportions of solved tasks using self-evaluation techniques of (Heuristic) and (GPT) for binary classification. (b) Classification of AlfWorld trajectories by reason of failure.

How to Add Capabilities Using RL?

Google Research

- **Goal:** Add a new capability to the model (e.g., calling a new tool) via RL
- **Issue:** The initial probability that the model will call this tool is very low (sparse exploration)
- **Solution A — Behavior Cloning:**
  - Show the model how it needs to do it (create SFT data for using this tool)
- **Solution B — Instruction Injection:**
  - Tell the model how to use the tool in the prompt
  - Remove this (or apply probabilistically) during training
  - Make sure the injected instructions are not propagated in the generations (thoughts, etc.)
- **Solution C — Hinting:**
  - Add a hint to elicit the required behavior, then remove the hint when using successful rollouts for SFT/RL

Scaling Laws and RL

Google Research

- **In pre-training we have scaling laws**
  - *(Scaling Laws for Neural Language Models, Kaplan et al.)*
  - This allows testing model architecture changes on smaller models
- **In RL, the “loss” is on-policy**
  - Model capability affects the task we can work with
  - Need to scale the tasks while scaling the model
  - Harder to generalize/extrapolate
  - E.g., hard to work on a very long-horizon problem using small academic models

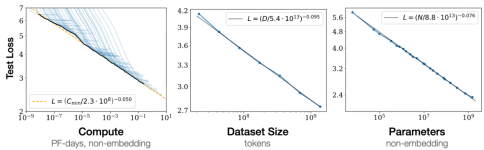


Figure 1 Language modeling performance improves smoothly as we increase the model size, datasetset size, and amount of compute[2] used for training. For optimal performance all three factors must be scaled up in tandem. Empirical performance has a power-law relationship with each individual factor when not bottlenecked by the other two.

Open Research Questions

Google Research

- **Credit Assignment Problem**
  - How to make a value LLM?
  - Can we have a textual value?
- **RL and Generalization**
  - How does the curriculum affect learning?
  - Should we complete a task, or build on similar tasks in the mixture?
- **Long Horizon**
- **Multi-agent**
  - Parallel Credit Assignment — How is the credit spread across agents?
  - Multi-agent value function?